# Check-in 2

**Forging the Watermark: Benchmarking False Attribution in Semantic Watermarks for Diffusion Models**

CMPE 261 Sec 01, Fall 2026. Group 6. Team: Kanaka Sarat Siripurapu (019132776), Latha Boralingaiah (018301361), Yashashav Devalapalli Kamalraj (017856371).

Repository: https://github.com/yashashav-dk/cmpe261-watermark-forgery

Shared Drive folder: https://drive.google.com/drive/folders/1wkXOxPjfI648JPod1Fi2m9ItfrPvWVIT

The instructor and the TA were invited to both before Check-in 1 and still have access.

## 1. Where each requirement is met

| Requirement | Where |
|---|---|
| Tentative abstract (recommended) | Section 7.1, updated with this check-in's results |
| Report outline (recommended) | Section 7.2 |
| Literature survey (recommended) | Section 7.3 |
| Approach (required) | Section 5: the pipeline from Check-in 1, now run on all six core schemes |
| Algorithms (recommended) | Section 6: imprint forgery, DDIM inversion, the six detectors |
| Meaningful check-ins (required) | Section 8.1 and the repository history |
| Contribution of each member since Check-in 1 | Section 8.2 |
| Repository and Drive link (required) | Top of this document |
| New results | Section 3; run logs in `results/checkin2/`, run notebook `notebooks/02_checkin2_colab.ipynb` |

## 2. Where Check-in 1 left off

Check-in 1 ran the full pipeline on two schemes, Tree-Ring and Gaussian Shading, and ended with this list of next steps, in the order fixed by the specification:

| # | Next step from Check-in 1 | Status at Check-in 2 |
|---|---|---|
| 1 | Clean baseline and real-photo baseline for the six core schemes | **Done for all six** at 50 images per cell (Section 3.1). Ring-ID, SFW, WIND and PRC were run for this check-in. The spec's full size (100 / 200 / 100) is still to do. |
| 2 | Imprint forgery with a same-model attacker for the six core schemes, 10 covers each, at fixed step budgets | **Done for all six at 5 covers**, with the same stop-at-first-detection protocol as Check-in 1 so the schemes compare directly (Section 3.2). 10 covers and fixed budgets are still to do. |
| 3 | Removal robustness under the four attacks MarkDiffusion implements | Not started |
| 4 | Imprint forgery with a different Stable Diffusion model as the attacker | Not started |

Everything except the four new schemes was kept the same as in Check-in 1: same model, same prompts, same 50 real photographs, same five cover photographs, same attack code, and the detector checked every 10 steps. The only other change is the GPU (an A100 instead of the RTX PRO 6000), which changes the cost per step but not the outputs. That makes the Check-in 2 numbers directly comparable with the Check-in 1 numbers, and Section 3 puts all six schemes side by side.

### Data

No new data was added. The four new schemes reuse the committed selections from Check-in 1:

| Source | Use | Size used |
|---|---|---|
| Stable-Diffusion-Prompts, eval split (bundled with MarkDiffusion) | Prompts for generated images | 50 prompts, same as Check-in 1 |
| MS-COCO val2017 | Real photographs: forgery covers and real-photo negatives | 50 photographs; the first 5 are the covers |
| Stable Diffusion 2.1-base, community mirror pinned by commit | Target model and same-model attacker | one model |

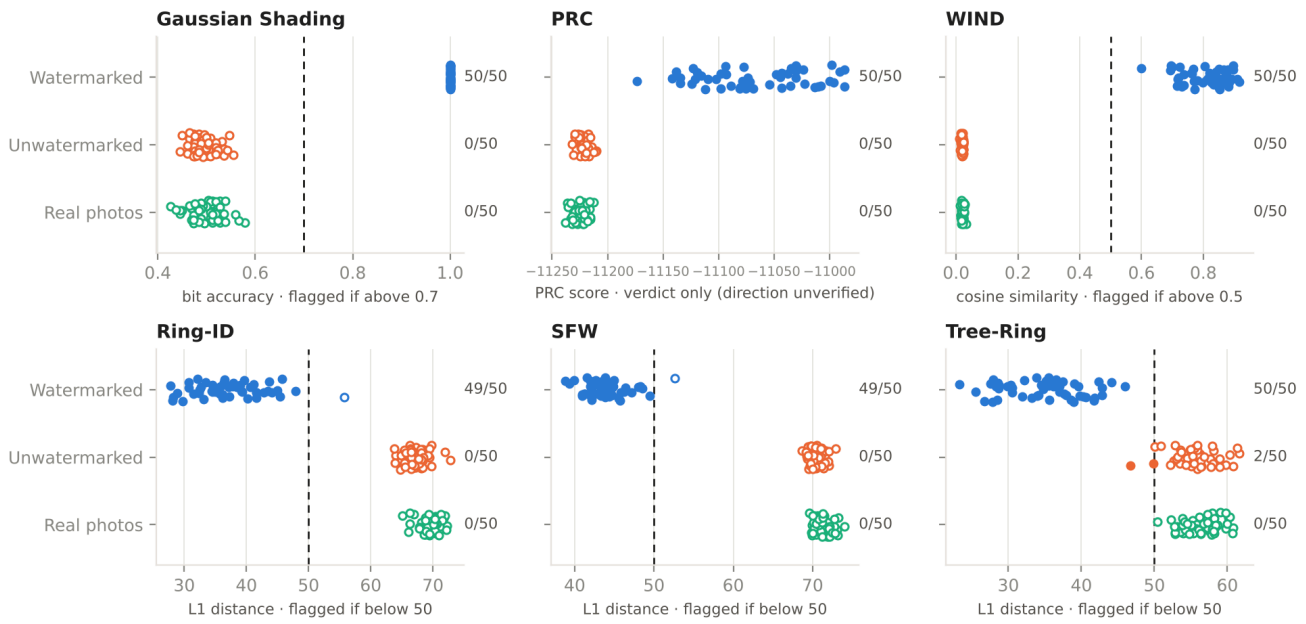
## 3. Results

Stable Diffusion 2.1-base at 512×512, 50 sampling steps, guidance 7.5, full precision, each scheme at its MarkDiffusion default configuration. Check-in 2 ran on one NVIDIA A100 40 GB (Colab). Brackets are 95% Wilson intervals. The full log is `results/checkin2/runs.csv`; the Check-in 1 rows come from `results/checkin1/` and `results/tr150/`.

### 3.1 Clean baseline and real-photo baseline

| Scheme | Detector score | TPR, watermarked | FPR, unwatermarked | FPR, real photos | Passes inclusion rule |
|---|---|---|---|---|---|
| Gaussian Shading* | bit accuracy, above 0.7 | 50/50 = 100% [93%, 100%] | 0/50 = 0% [0%, 7%] | 0/50 = 0% [0%, 7%] | yes |
| Tree-Ring* | L1 distance, below 50 | 50/50 = 100% [93%, 100%] | 2/50 = 4% [1%, 13%] | 0/50 = 0% [0%, 7%] | yes |
| Ring-ID | L1 distance, below 50 | 49/50 = 98% [90%, 100%] | 0/50 = 0% [0%, 7%] | 0/50 = 0% [0%, 7%] | yes |
| SFW | L1 distance, below 50 | 49/50 = 98% [90%, 100%] | 0/50 = 0% [0%, 7%] | 0/50 = 0% [0%, 7%] | yes |
| WIND | cosine similarity, above 0.5 | 50/50 = 100% [93%, 100%] | 0/50 = 0% [0%, 7%] | 0/50 = 0% [0%, 7%] | yes |
| PRC | decoder verdict | 50/50 = 100% [93%, 100%] | 0/50 = 0% [0%, 7%] | 0/50 = 0% [0%, 7%] | yes |

* From Check-in 1. Shaded rows are new. The inclusion rule from the specification is TPR ≥ 90% and FPR ≤ 5% on the clean baseline.




*Every detector score from the two baselines, one dot per image. A filled dot means the detector flagged the image as watermarked. The dashed line is the default threshold. PRC is plotted by raw score but judged by its decoder's verdict, because the direction of its score is not verified.*

**All six core schemes enter the benchmark.** Ring-ID and SFW each missed one watermarked image (scores 55.7 and 52.6 against a threshold of 50). None of the four new detectors flagged an unwatermarked generation or a real photograph.

**Ring-ID and SFW leave much more room than Tree-Ring.** The three schemes use the same kind of score, an L1 distance with a threshold of 50. Real photographs scored 50.4 to 60.8 under Tree-Ring, so a real photograph started only a few units from being flagged. Under Ring-ID they scored 65.1 to 72.3, and under SFW 69.6 to 74.1. WIND separates even more clearly: watermarked images scored 0.60 to 0.92 and real photographs 0.013 to 0.032, against a threshold of 0.5.

### 3.2 Imprint forgery on the four new schemes

The attack is the Check-in 1 imprint forgery, unchanged. Each scheme gets five watermarked references (one per cover), and each cover is optimized until the detector flags it or 150 steps pass.

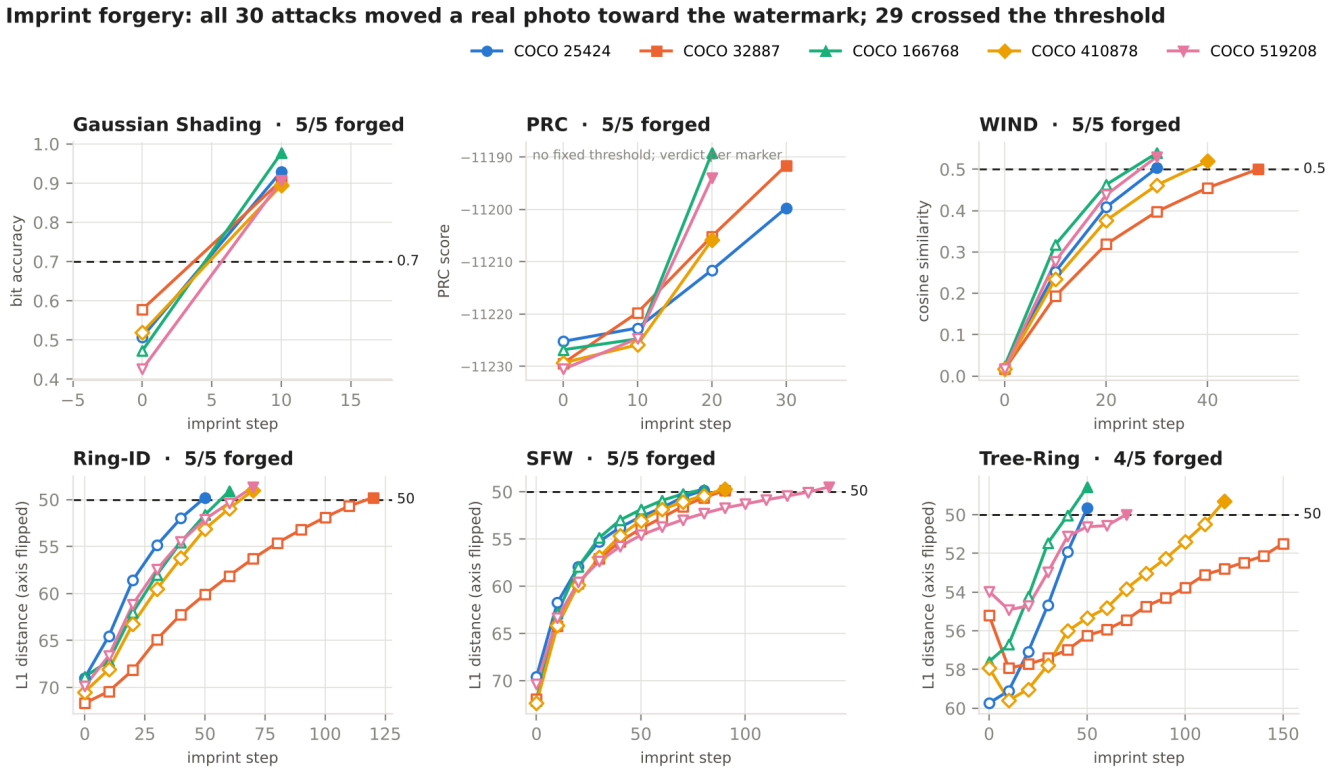
| Scheme | Forgery success (imprint, C1) | Median first-detection step | PSNR / SSIM / LPIPS at last step, median | Seconds per imprint step |
|---|---|---|---|---|
| Gaussian Shading* | 5/5 = 100% [57%, 100%] | 10 | 27.5 dB / 0.737 / 0.079 | 4.9 |
| PRC | 5/5 = 100% [57%, 100%] | 20 | 27.7 dB / 0.671 / 0.118 | 8.4 |
| WIND | 5/5 = 100% [57%, 100%] | 30 | 27.0 dB / 0.647 / 0.148 | 8.4 |
| Ring-ID | 5/5 = 100% [57%, 100%] | 70 | 26.0 dB / 0.602 / 0.194 | 8.4 |
| SFW | 5/5 = 100% [57%, 100%] | 90 | 25.1 dB / 0.573 / 0.222 | 8.4 |
| Tree-Ring* | 4/5 = 80% [38%, 96%] (150-step run) | 60 (of the four forged) | 24.2 dB mean (Check-in 1) | 4.9 |

* From Check-in 1. Rows are ordered from easiest to hardest to forge. Seconds per step differ because Check-in 1 ran on an RTX PRO 6000 and Check-in 2 on an A100.

Per cover, the step at which the detector first flagged the image, the score at that step, and the PSNR of the forgery:

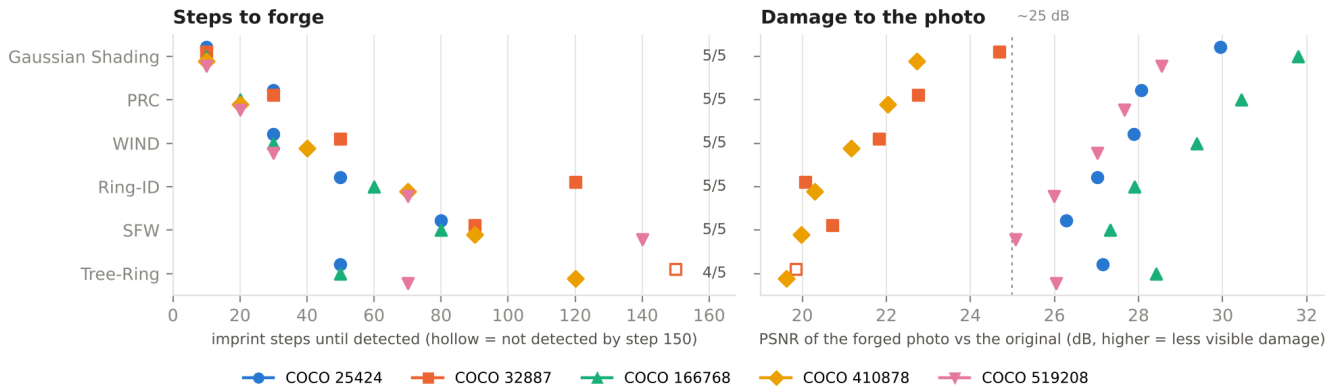
| Cover (COCO id) | PRC | WIND | Ring-ID | SFW | Tree-Ring* (150-step run) |
|---|---|---|---|---|---|
| 410878 | step 20, 22.0 dB | step 40 (0.519), 21.2 dB | step 70 (49.01), 20.3 dB | step 90 (49.69), 20.0 dB | step 120 (49.3), 19.6 dB |
| 25424 | step 30, 28.1 dB | step 30 (0.503), 27.9 dB | step 50 (49.78), 27.0 dB | step 80 (49.85), 26.3 dB | step 50 (49.7), 27.2 dB |
| 166768 | step 20, 30.5 dB | step 30 (0.539), 29.4 dB | step 60 (49.12), 27.9 dB | step 80 (49.75), 27.3 dB | step 50 (48.6), 28.4 dB |
| 32887 | step 30, 22.8 dB | step 50 (0.500), 21.8 dB | step 120 (49.78), 20.1 dB | step 90 (49.87), 20.7 dB | not flagged, 51.5 at step 150, 19.9 dB |
| 519208 | step 20, 27.7 dB | step 30 (0.530), 27.0 dB | step 70 (48.66), 26.0 dB | step 140 (49.49), 25.1 dB | step 70 (49.99), 26.0 dB |

Scores in brackets: cosine similarity for WIND (flagged above 0.5), L1 distance for Ring-ID, SFW and Tree-Ring (flagged below 50). PRC is reported by verdict only.




*Detector score for each cover photograph as the imprint proceeds, for all six schemes. Step 0 is the untouched photograph. Every panel is turned so that up means closer to being flagged. A filled marker means the detector flagged the image. Gaussian Shading and Tree-Ring are from Check-in 1.*

**Wider detector margins slow forgery down but do not stop it**




*Left: imprint steps until the detector first flagged each cover (hollow = never flagged within 150 steps). Right: PSNR of the final image against the original photograph.*

## What the numbers show

**Every core scheme was forged.** All 20 new attacks succeeded. Across both check-ins, 29 of 30 attacks turned a real photograph into one the provider's detector calls watermarked. The only miss is one Tree-Ring cover, which was still moving toward the threshold at 150 steps.

**The order is not what the scheme papers would suggest.** PRC, which comes with a proof that its outputs cannot be told apart from unwatermarked ones, fell in 20 to 30 steps. WIND, which salts its noise so there is no fixed pattern, fell in 30 to 50. Neither property protects against an attacker who copies the inverted noise of a real watermarked image, which is exactly what the imprint attack does.

**A bigger margin slows the attack down but does not stop it.** Ring-ID and SFW start about 15 to 24 units from their threshold, against 0.4 to 11 for Tree-Ring. They needed 3 to 4 times as many steps as PRC or WIND, and still fell on every cover. Margin is not the whole story, though: Tree-Ring starts closest to its threshold and is the only scheme that resisted on a cover.

**The Ring-ID and SFW verdicts are close calls.** Their forgeries were flagged at scores of 48.7 to 49.9 against a threshold of 50. In Check-in 1, repeated Tree-Ring runs differed by about 0.3 near the threshold, so some of these verdicts could flip on a rerun. This is the same problem that made us move to fixed step budgets for the full benchmark.

**Damage depends on the photograph more than on the scheme.** Covers 410878 and 32887 ended at 20 to 23 dB under every scheme except Gaussian Shading, which is visible texture damage. Covers 166768 and 25424 stayed at 26 to 31 dB. Cover 32887 was also the slowest for Ring-ID and WIND and the one Tree-Ring never gave up.

## Limits of this run

- Five covers per scheme still give wide intervals: 5/5 is 57% to 100% at 95% confidence.
- Each attack stopped at the first detection. That uses the detector's verdict, which a real attacker would not have. We kept it only so the new schemes are measured exactly like the Check-in 1 schemes.
- The baselines used 50 images per cell, half of the specification's sizes, so the calibrated 1% false-positive operating point is not computed yet. Every rate here is at the default threshold.
- The attacker used the provider's own model. This is the strongest case and an upper bound for the other attacker models.

### Cost, and what it implies

One imprint step took 8.4 seconds on the A100 (8.38 to 8.41 across the four schemes), against 4.9 seconds on the RTX PRO 6000 in Check-in 1. One generation plus detection took about 7 seconds, and detection alone on a real photograph about 3.5 to 4.5 seconds. This check-in used about 1.0 GPU-hour for the baselines and 2.7 GPU-hours for the forgery runs.

The full F2 item (six schemes, 10 covers, run to 150 steps without stopping) is 9,000 imprint steps: about 21 A100-hours, or about 12 hours on the Check-in 1 GPU. That is the largest cost left in the floor, and it fits in Colab quota only if it is split over several sessions. The CSV log already supports that.

### Problems found by running it

A two-image dry run per scheme was done before the long runs. It caught three missing packages that MarkDiffusion does not install. Without them SFW and PRC fail at import:

| Scheme | Error | Fix |
|---|---|---|
| SFW | `No module named 'qrcode'` | `pip install qrcode` |
| PRC | `No module named 'galois'` | `pip install galois` |
| PRC | `No module named 'ldpc'` | `pip install "ldpc<2"`; version 2 renamed the `bp_decoder` that PRC imports |

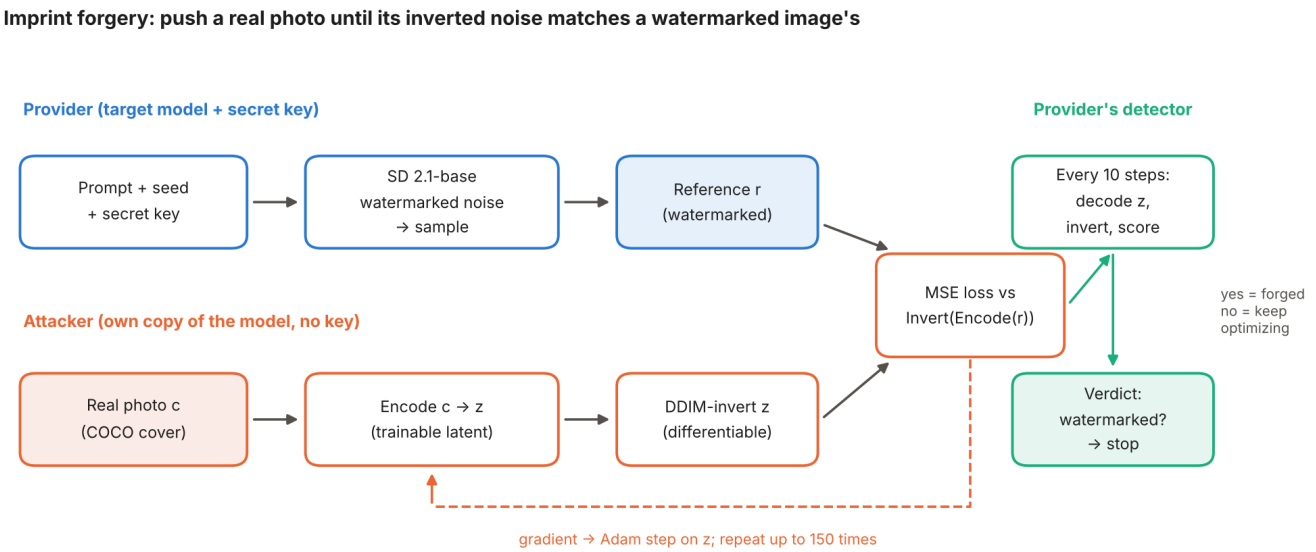
The fix is in the install cell of `notebooks/02_checkin2_colab.ipynb`. Two smaller things also came up. The Colab copy of the repository was still on the `checkin-1` branch from an earlier session, so the notebook now deletes and re-clones on `main`. And a T4 GPU was about 8 times slower than the A100 (27 s against 3.5 s per generation), so the notebook prints the GPU name first.

## 4. Software added

| Path | Purpose |
|---|---|
| `results/checkin2/runs.csv` | 755 rows: 600 baseline detections and 155 imprint records for Ring-ID, SFW, WIND and PRC |
| `results/checkin2/forgery_per_cover.csv` | One row per scheme and cover: steps, verdict, PSNR (all six schemes) |
| `results/checkin2/figures/` | The three figures in Section 3 |
| `scripts/checkin2_figures.py` | Draws the figures from the Check-in 1 and Check-in 2 logs; runs on a CPU in a few seconds |
| `notebooks/02_checkin2_colab.ipynb` | The Check-in 2 run on Colab, kept with its outputs |

No change was needed in `src/wmforge/`. The harness already declared the score key and direction for all six schemes, so the new schemes were added from the command line (`--schemes RI SFW WIND PRC`). This was the point of keeping the harness and the attack separate in Check-in 1.

## 5. Approach




*The imprint forgery as implemented. The provider makes one watermarked reference with its key. The attacker, who has the model but not the key, changes a real photograph until its inverted noise matches the reference's. The provider's detector is only used to record the outcome.*

**Framing.** We treat the problem as a measurement task, not a new attack. The pipeline has three parts that only pass images to each other, so a new scheme or a new attack can be added on one side without touching the other.

- **Harness** (`src/wmforge/harness.py`) wraps MarkDiffusion. For a scheme it generates a watermarked or unwatermarked image from a prompt and a seed, and runs the scheme's detector on any image, returning the raw score and the verdict. MarkDiffusion samples a scheme's starting noise once at load, so the harness reseeds and redraws the initial latents for every image.
- **Attack** (`src/wmforge/imprint.py`) takes a cover photograph, a watermarked reference and an attacker pipeline, and returns the forged image and a per-step record. It is adapted from the official code of [16] and rewritten against MarkDiffusion's dependencies so one environment runs everything.
- **Metrics and provenance** (`metrics.py`, `records.py`, `report.py`) compute rates with Wilson intervals, continuous metrics with bootstrap intervals, and PSNR, SSIM and LPIPS. They write one CSV row per (scheme, attack, condition, image, step) with the seed and model commit hash, so an interrupted run resumes from the log.

**Approaches explored, 1: attack code.** Running the reference implementation as is failed because its pinned packages conflict with MarkDiffusion's. Re-implementing the attack against MarkDiffusion's environment worked and is what the repository contains. The GPL-3.0 license of the original is kept.

**Approaches explored, 2: run protocol.** Stopping at the first detection, with a cap, is cheap and gives a median first-detection step. But it uses the detector's verdict, which a real attacker does not have, and verdicts near the threshold flip between runs. Running to a fixed step budget and reporting success at several budgets costs more but matches the threat model. Both check-ins use the first protocol so the six schemes can be compared; the benchmark proper uses the second. The 150-step Tree-Ring rerun showed the flip: one cover scored 50.02 at step 50 in one run and 49.67 in the next.

**Approaches explored, 3: precision.** Half precision halves memory, but Gaussian Shading builds float32 latents that a half-precision pipeline rejects. The target pipeline and the attack therefore run in full precision with gradient checkpointing.

**Approaches explored, 4: dependencies.** Running the four new schemes showed that MarkDiffusion's own dependency list is not enough for SFW and PRC (Section 3, Problems found by running it). A dry run with two images per scheme is now the first step for any new scheme.

## 6. Algorithms

**Imprint forgery, adapted from [16].** The attacker has the reference image, its own diffusion model, and nothing about the scheme. It moves the cover's latent until its own inversion of the cover matches its inversion of the reference.

```
Input: cover image c, watermarked reference r, attacker pipeline (VAE, UNet,
       DDIM inverse scheduler), step cap K
1. z_target = Invert(Encode(r))      # no gradient; 50 DDIM inversion steps,
                                     # empty prompt, no guidance
2. z = Encode(c), made a trainable parameter; Adam, learning rate 1e-2
3. for step in 1..K:
       loss = MSE(Invert(z), z_target)   # differentiable inversion; UNet
                                         # activations recomputed on backward
       backprop, Adam step
       every 10 steps and at K: image = Decode(z); run the provider's
                                detector; record score and verdict
4. return Decode(z) and the per-step records
```

**DDIM inversion [12]** is used by every detector and by the attack. Starting from the clean latent, the DDIM update is applied in the direction of increasing noise for 50 steps, using the model's noise prediction with an empty prompt. DDIM is deterministic, so inverting and then generating gives back almost the same image. That is what makes the recovered noise usable as a watermark channel, and also what the attack exploits.

**The six detectors.** All six invert the image to its starting noise and compare it with the key. They differ in what they compare and how the score is read. The thresholds are MarkDiffusion's defaults at the pinned commit; every verdict in our logs agrees with them.

| Scheme | What the detector compares | Score | Flagged when |
|---|---|---|---|
| Tree-Ring [1] | Ring of Fourier coefficients of the recovered noise against the key | L1 distance | below 50 |
| Ring-ID [2] | Multi-key version of the Tree-Ring ring | L1 distance | below 50 |
| SFW | Fourier-domain pattern against the key, as implemented in MarkDiffusion [9] | L1 distance | below 50 |
| Gaussian Shading [3] | Signs of the recovered noise read back as bits, against the key | bit accuracy | above 0.7 |
| WIND [5] | Recovered noise against the salted noise family | cosine similarity | above 0.5 |
| PRC [4] | Decoded pseudorandom codeword | PRC score | decoder verdict |

**Statistics.** Rates use 95% Wilson intervals, which stay informative when every trial succeeds or fails. Continuous metrics use 10,000-resample percentile bootstraps. The calibrated operating point will set each scheme's threshold to a 1% false-positive rate on 200 unwatermarked generations.

## 7. Draft of the report

### 7.1 Tentative abstract

A semantic watermark hides a signal in the starting noise of a diffusion model and checks for it by inverting a candidate image back to that noise. Published evaluations of these schemes test almost only whether the signal survives edits. We study the opposite failure, false attribution: an attacker takes a real photograph that the

provider never generated and changes it until the provider's detector says it is watermarked. Building on the MarkDiffusion toolkit and the black-box imprint attack of Müller et al., we benchmark six core semantic schemes, and up to three more, on Stable Diffusion 2.1-base. For each scheme we report the share of real photographs the detector accepts after the attack, at the toolkit's default threshold and at a threshold set to a 1% false-positive rate, with the attacker using either the provider's model or a different one. Resistance to removal is reported next to forgeability so the two can be compared on the same schemes. With the provider's model as the attacker, all six core schemes have been forged so far: Gaussian Shading, PRC, WIND, Ring-ID and SFW on 5 of 5 photographs and Tree-Ring on 4 of 5, within 10 to 140 optimization steps. All experiments are inference only, and every rate is reported with a confidence interval.

### 7.2 Report outline

1. **Introduction.** Removal versus forgery; why false attribution matters for provenance; contributions.
2. **Background and related work.** Semantic watermarking and inversion-based detection, the nine schemes, removal attacks, forgery attacks, existing benchmarks (Section 7.3).
3. **Threat model.** Attacker knowledge and compute, what counts as a successful forgery, the three attacker-model conditions.
4. **Benchmark design.** Attacks added to MarkDiffusion; scheme inclusion rule; metrics; detector operating points; step budgets.
5. **Experimental setup.** Models, data, sample sizes, statistics, compute.
6. **Results.** Clean baseline; false attribution by scheme and threshold; transfer across attacker models; removal robustness versus forgeability; perceptual cost of forgery.
7. **Discussion.** Which design properties track forgery resistance; implications for provenance systems; limitations.
8. **Conclusion and future work.**
9. **References.**
10. **Appendix.** Scheme configurations, per-scheme results, excluded schemes, reproducibility details.

### 7.3 Literature survey

**Schemes.** Semantic watermarks sit in the latent noise rather than in the pixels. Tree-Ring [1] writes a ring pattern into the Fourier transform of the initial noise and detects it by DDIM inversion [12] of the image followed by an L1 distance to the pattern. RingID [2] shows that part of Tree-Ring's apparent robustness comes from the distribution shift the pattern introduces, and redesigns the key so that many keys can be told apart. Gaussian Shading [3] maps a secret bit string to the initial noise through Gaussian quantiles, which keeps the noise distribution unchanged and lets the detector recover the bits and score bit accuracy. PRC [4] replaces the bit string with a pseudorandom error-correcting code and proves that watermarked and unwatermarked outputs are computationally indistinguishable. WIND [5] combines a Fourier group index with a salted family of initial noises, so retrieval is fast and the noise carries no fixed pattern. ROBIN [6] implants the watermark at an intermediate diffusion state and optimizes a hiding prompt so the final image shows no artifact. SEAL [7] ties the watermark to a caption of the image, so the key must agree with what the image shows. GaussMarker [8] embeds in both the spatial and the frequency domain and adds a learned restorer for noise recovered from edited images. MarkDiffusion [9] puts these schemes behind one interface with shared detection code, which is what makes a cross-scheme benchmark possible in one semester.

**Removal.** The removal literature is large. WAVES [10] is the reference benchmark; it ranks watermarks under distortions, diffusion-based regeneration and adversarial attacks. The four distortion and purification attacks we use are the ones MarkDiffusion ships. Zhao et al. [11] show that regeneration through a diffusion model removes any pixel-space watermark of bounded strength, and DiffPure [13] is the purification procedure most of these attacks reuse. UnMarker [14] attacks the spectral content directly and reaches semantic watermarks too. Saberi et al. [15] give the first fundamental limits for watermark-based detectors and include a spoofing attack, the earliest forgery result we found.

**Forgery.** Müller et al. [16] is the direct ancestor of this project. Their imprint attack optimizes a cover image so that the attacker's inversion of it lands on the inverted noise of one watermarked reference, with no knowledge of the scheme; their reprompting attack regenerates from an inverted reference under a new prompt. They evaluate Tree-Ring and Gaussian Shading only. Jain et al. [17] forge and remove latent-noise watermarks from a single image and cover more schemes. This partly answers the open item in our specification: forgery evaluations now exist beyond the two original schemes, but not on a common model, threshold policy or step budget, and not next to removal numbers. That is the gap this benchmark fills.

**What the survey shows.** None of the scheme papers reports a false-attribution rate on real photographs. Benchmarks report removal only. Forgery results exist for a few schemes under different protocols. A benchmark that reports forgeability and removal robustness on the same schemes, the same model and the same thresholds is missing. Section 3.2 is a first step: Ring-ID, SFW, WIND and PRC under the same attack, photographs and model as Tree-Ring and Gaussian Shading.

## 8. Check-ins and contributions

### 8.1 Commits since Check-in 1

The repository has 29 commits on `main` by all three members. Each commit adds one component, fix, result or document. The eleven commits since Check-in 1 are:

| Date | Member | Commit |
|---|---|---|
| 2026-10-18 | Kanaka Sarat | docs: add Check-in 2 report |
| 2026-10-18 | Kanaka Sarat | feat: add script that draws the Check-in 2 figures from the run logs |
| 2026-10-18 | Yashashav | docs: update README findings and status for Check-in 2 |
| 2026-10-18 | Yashashav | results: add Check-in 2 figures across the six core schemes |
| 2026-10-18 | Latha | feat: add Check-in 2 Colab notebook with run outputs |
| 2026-10-18 | Latha | results: Check-in 2 baselines and imprint forgery for RI, SFW, WIND, PRC |
| 2026-10-05 | Kanaka Sarat | docs: list member IDs and emails in check-in 2 |
| 2026-10-05 | Kanaka Sarat | docs: add check-in 2 with literature survey, approach, and algorithms |
| 2026-10-02 | Yashashav | docs: link the shared Drive folder in the README |
| 2026-10-02 | Latha | docs: report findings so far in the README; add 150-step follow-up to check-in 1 |
| 2026-10-02 | Yashashav | results: add Tree-Ring imprint run with a 150-step cap |

The 18 commits before these (2026-10-01) built the Check-in 1 pipeline and are listed in the Check-in 1 report. Commits per member in total: Kanaka Sarat 10, Yashashav 10, Latha 9.

### 8.2 What each member did since Check-in 1

**Latha Boralingaiah** (attack implementation and transferability). Ran all of the Check-in 2 experiments on Colab. Wrote the dry run for the four new schemes and traced the SFW and PRC import failures to three missing packages (`qrcode`, `galois`, `ldpc<2`), including the `ldpc` version pin. Ran the F1 baselines for Ring-ID, SFW, WIND and PRC (600 detections, about 1 A100-hour) and the F2 imprint forgery on 20 covers (about 2.7 A100-hours), handling the Colab disconnects and the stale `checkin-1` checkout. Committed the run log and the run notebook with its outputs. Earlier, on 2026-10-02, wrote the README findings section and the 150-step follow-up in the Check-in 1 write-up.

**Yashashav Devalapalli Kamalraj** (generation harness and environment). Ran the Tree-Ring imprint rerun with the cap raised to 150 steps (2026-10-02), which raised Tree-Ring from 2 of 5 to 4 of 5 forged and showed the verdict flip near the threshold that motivates the fixed-budget protocol. Set up and linked the shared Drive folder. For Check-in 2, committed the six-scheme figures and updated the README findings table and status against the specification.

**Kanaka Sarat Siripurapu** (metrics, statistics and provenance). Wrote the literature survey (17 papers), the approach and the algorithm sections, and the first Check-in 2 draft (2026-10-05). For Check-in 2, committed the figure script, which recomputes the per-scheme medians and intervals from the logs, and the final report.

Each member committed their own work from their own GitHub account.

## 9. Next steps

In the order fixed by the specification:

1. Bring the clean and real-photo baselines to full size (100 watermarked, 200 unwatermarked, 100 real per scheme) and compute the 1% false-positive thresholds. About 3 A100-hours.
2. Imprint forgery with a same-model attacker for the six core schemes, 10 covers each, run to fixed budgets of 50, 100 and 150 steps without stopping at detection. About 21 A100-hours, split across sessions.
3. Removal robustness under the four attacks MarkDiffusion implements (JPEG, blur, crop-and-scale, diffusion purification), 100 watermarked images per scheme and attack.
4. Imprint forgery with Stable Diffusion 1.5 as the attacker (condition C2).

**Risks.** The main risk is still Colab quota. Runs resume from the CSV log, and if time runs short the step cap is lowered before the cover count, as the specification says. A scheme that fails the inclusion rule at full sample size is reported as excluded. CoE HPC access is still unconfirmed, and no floor item depends on it.

## References


[1] Y. Wen, J. Kirchenbauer, J. Geiping, T. Goldstein. Tree-Ring Watermarks: Fingerprints for Diffusion Images that are Invisible and Robust. NeurIPS 2023. arXiv:2305.20030.

[2] H. Ci, P. Yang, Y. Song, M. Z. Shou. RingID: Rethinking Tree-Ring Watermarking for Enhanced Multi-Key Identification. ECCV 2024. arXiv:2404.14055.

[3] Z. Yang et al. Gaussian Shading: Provable Performance-Lossless Image Watermarking for Diffusion Models. CVPR 2024. arXiv:2404.04956.

[4] S. Gunn, X. Zhao, D. Song. An Undetectable Watermark for Generative Image Models. 2024. arXiv:2410.07369.

[5] K. Arabi, B. Feuer, R. T. Witter, C. Hegde, N. Cohen. Hidden in the Noise: Two-Stage Robust Watermarking for Images. 2024. arXiv:2412.04653.

[6] H. Huang, Y. Wu, Q. Wang. ROBIN: Robust and Invisible Watermarks for Diffusion Models with Adversarial Optimization. NeurIPS 2024. arXiv:2411.03862.

[7] K. Arabi, R. T. Witter, C. Hegde, N. Cohen. SEAL: Semantic Aware Image Watermarking. 2025. arXiv:2503.12172.

[8] K. Li et al. GaussMarker: Robust Dual-Domain Watermark for Diffusion Models. ICML 2025. arXiv:2506.11444.

[9] L. Pan et al. MarkDiffusion: An Open-Source Toolkit for Generative Watermarking of Latent Diffusion Models. 2025. arXiv:2509.10569.

[10] B. An et al. WAVES: Benchmarking the Robustness of Image Watermarks. ICML 2024.

[11] X. Zhao et al. Invisible Image Watermarks Are Provably Removable Using Generative AI. NeurIPS 2024. arXiv:2306.01953.

[12] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. ICLR 2021. arXiv:2010.02502.

[13] W. Nie et al. Diffusion Models for Adversarial Purification. ICML 2022. arXiv:2205.07460.

[14] A. Kassis, U. Hengartner. UnMarker: A Universal Attack on Defensive Image Watermarking. IEEE S&P 2025. arXiv:2405.08363.

[15] M. Saberi et al. Robustness of AI-Image Detectors: Fundamental Limits and Practical Attacks. ICLR 2024. arXiv:2310.00076.

[16] A. Müller, D. Lukovnikov, J. Thietke, A. Fischer, E. Quiring. Black-Box Forgery Attacks on Semantic Watermarks for Diffusion Models. CVPR 2025. arXiv:2412.03283.

[17] A. Jain, Y. Kobayashi, N. Murata, Y. Takida, T. Shibuya, Y. Mitsufuji, N. Cohen, N. Memon, J. Togelius. Forging and Removing Latent-Noise Diffusion Watermarks Using a Single Image. 2025. arXiv:2504.20111.